

URL SHORTENER BACKEND SYSTEM

A backend system built using Node.js and MongoDB that converts long URLs into short, shareable links and redirects users to the original URL when accessed.

BY
JAGADEESH MUTHAMSETTY

1. Project Overview

This project is a **URL Shortener Backend System** built using Node.js and MongoDB.

It converts long URLs into short, unique URLs and redirects users back to the original URL when the short link is accessed.

Example: <https://google.com> → <http://localhost:3000/av121nD>

2. Problem Statement

Long URLs are:

- Hard to share
- Not user-friendly
- Difficult to remember

This system solves that by generating:

- Short, unique links
- Easy redirection to original URLs

3. Tech Stack

- Node.js → runtime environment
- Express.js → backend framework
- MongoDB Atlas → database
- Mongoose → database modeling
- Nanoid → unique ID generator
- dotenv → environment management

Setup Instructions:

- Install [Node.js](#)
- Install MongoDB Atlas or local MongoDB
- Clone the repository using git clone
- Run `npm install` to install dependencies
- Create a `.env` file and add MongoDB connection string
- Start the server using `npm run dev`

4. Project Architecture

Client (Postman / Browser)



Express Server (API)



Controller (Logic)



MongoDB Database



Response / Redirect

5. PROJECT STRUCTURE

src/

├── app.js

├── config/db.js

├── routes/urlRoutes.js

├── controllers/urlController.js

├── models/urlModel.js

└── services/urlService.js

server.js

6. HOW THE SYSTEM WORKS

User sends a long URL



Server generates a unique short ID (nanoid)



Stores mapping in MongoDB



Returns a short URL to the user



User opens short URL



Server fetches original URL



Redirects the user to the original website

7. API ENDPOINTS

7.1 Create Short URL

Endpoint:

POST /shorten

Request Body:

```
{
  "originalUrl": "https://google.com"
}
```

Response:

```
{
  "shortUrl": "http://localhost:3000/avI21nD"
}
```

7.2 Redirect URL

Endpoint:

GET /:shortId

Example:

http://localhost:3000/avI21nD

Function:

Redirects to the original URL stored in the database.

7.3 Home Route

GET /

Response:

The URL Shortener API is running

8. KEY FEATURES

URL shortening system

Unique ID generation using Nanoid

MongoDB data storage

Fast URL redirection

REST API architecture

9. CORE CODE SNIPPETS

9.1 Server Setup

</>Javascript code

```
const dotenv = require("dotenv");
dotenv.config();

const app = require("./src/app");
const connectDB = require("./src/config/db");

connectDB();

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

9.2 Create Short URL Logic

</>Javascript code

```
const { nanoid } = require("nanoid");

const shortId = nanoid(7);
```

9.3 Save to Database

</>Javascript code

```
await Url.create({
  originalUrl,
  shortId
});
```

9.4 Redirect Logic

</>Javascript code

```
const url = await Url.findOne({ shortId });

if (!url) {
  return res.status(404).json({ error: "URL not found" });
}

res.redirect(url.originalUrl);
```

10. TOOLS USED FOR TESTING

Postman

curl (Terminal)

Browser

Example:

</>Terminal/Bash

```
curl -X POST http://localhost:3000/shorten \
-H "Content-Type: application/json" \
-d '{"originalUrl":"https://google.com"}'
```

11. OUTPUT EXAMPLE

Input:

https://google.com

Output:

http://localhost:3000/avl21nD

12. LEARNING OUTCOMES

Backend API development using Node.js

MongoDB database integration

REST API design principles

URL redirection logic

Server-side architecture understanding

CONCLUSION

This project demonstrates a real-world backend system similar to services like Bitly or TinyURL. It showcases skills in API development, database integration, and backend architecture design.